

Yulu - Hypothesis Testing

Problem statement and business objective

Business Problem

Yulu has experienced a decline in revenue and wants to understand which factors influence the demand for its shared electric cycles in the Indian market. The goal is to identify the variables that significantly affect demand and determine how well these variables explain variations in demand.

Business objective

- Identify the factors that influence bike rental demand.
- Measure the impact of these factors on demand.
- Provide recommendations to increase rentals and improve revenue.

Column profiling

- datetime: datetime
- season: season (1: spring, 2: summer, 3: fall, 4: winter)
- holiday: whether day is a holiday or not (extracted from <http://dchr.dc.gov/page/holiday-schedule>)
- Workingday: if day is neither weekend nor holiday is 1, otherwise is 0.
- Weather:
 1. Clear, Few clouds, partly cloudy, partly cloudy
 2. Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist
 3. Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds
 4. Heavy Rain + Ice Pallets + Thunderstorm + Mist, Snow + Fog
- temp: temperature in Celsius
- atemp: feeling temperature in Celsius
- humidity: humidity
- windspeed: wind speed
- casual: count of casual users
- registered: count of registered users
- count: count of total rental bikes including both casual and registered

EDA (Exploratory data analysis)

```
In [2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import statsmodels.api as sm
from statsmodels.stats.oneway import anova_oneway # Welch's Anova
from scipy import stats
```

Structure checking and characteristics of dataset

```
In [3]: df = pd.read_csv("bike_sharing.csv")
df.head()
```

```
Out[3]:
```

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395	81	0.0	3	13	16
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635	80	0.0	8	32	40
2	2011-01-01 02:00:00	1	0	0	1	9.02	13.635	80	0.0	5	27	32
3	2011-01-01 03:00:00	1	0	0	1	9.84	14.395	75	0.0	3	10	13
4	2011-01-01 04:00:00	1	0	0	1	9.84	14.395	75	0.0	0	1	1

```
In [4]: # Checking the shape of data
df.shape
```

```
Out[4]: (10886, 12)
```

```
In [5]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    datetime    10886 non-null  str
1    season      10886 non-null  int64
2    holiday     10886 non-null  int64
3    workingday  10886 non-null  int64
4    weather     10886 non-null  int64
5    temp        10886 non-null  float64
6    atemp       10886 non-null  float64
7    humidity    10886 non-null  int64
8    windspeed   10886 non-null  float64
9    casual      10886 non-null  int64
10   registered  10886 non-null  int64
11   count       10886 non-null  int64
dtypes: float64(3), int64(8), str(1)
memory usage: 1020.7 KB
```

As we can see only datetime column is Object we can change it to date time so that we can access the dates.

We will also change categorical columns to category which are wrongly mapped to **int64** such as **season**, **holiday**, **workingday**, **weather**

```
In [6]: # Conversion of datetime to correct format
df["datetime"] = pd.to_datetime(df["datetime"])
df["year"] = df["datetime"].dt.year
df["month_name"] = df["datetime"].dt.month_name()
month_order = ["January", "February", "March", "April", "May", "June",
               "July", "August", "September", "October", "November", "December"]
df["month_name"] = pd.Categorical(df["month_name"], categories=month_order, ordered=True)
df["hour"] = df["datetime"].dt.hour
```

```
In [7]: df.head()
```

```
Out[7]:
```

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count	year	month_name	hour
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395	81	0.0	3	13	16	2011	January	0
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635	80	0.0	8	32	40	2011	January	1
2	2011-01-01 02:00:00	1	0	0	1	9.02	13.635	80	0.0	5	27	32	2011	January	2
3	2011-01-01 03:00:00	1	0	0	1	9.84	14.395	75	0.0	3	10	13	2011	January	3
4	2011-01-01 04:00:00	1	0	0	1	9.84	14.395	75	0.0	0	1	1	2011	January	4

```
In [8]: # Converting season, holiday, workingday, weather to category datatype
df[["season", "holiday", "workingday", "weather"]] = df[["season", "holiday", "workingday", "weather"]].astype("category")
```

```
In [9]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    datetime    10886 non-null  datetime64[us]
1    season      10886 non-null  category
2    holiday     10886 non-null  category
3    workingday  10886 non-null  category
4    weather     10886 non-null  category
5    temp        10886 non-null  float64
6    atemp       10886 non-null  float64
7    humidity    10886 non-null  int64
8    windspeed   10886 non-null  float64
9    casual      10886 non-null  int64
10   registered  10886 non-null  int64
11   count       10886 non-null  int64
12   year        10886 non-null  int32
13   month_name  10886 non-null  category
14   hour        10886 non-null  int32
dtypes: category(5), datetime64[us](1), float64(3), int32(2), int64(4)
memory usage: 819.2 KB
```

As we can say columns data types have been changed to there correct types this will help us with our further analysis.

Moving forward lets change **season**, **holiday** and **workingday** from numbers to there proper names as already stated in the column profiling.

```
In [10]: # Converted season, holiday , workingday and weather categories to its proper names.
category_maps = {
    "season": {
        1: "spring",
        2: "summer",
        3: "fall",
        4: "winter"
    },
    "holiday": {
        0: "No",
        1: "Yes"
    },
    "workingday": {
        0: "No",
        1: "Yes"
    },
    "weather": {
        1: "clear",
        2: "misty",
        3: "light snow/rain",
```

```

        4: "heavy snow/rain"
    }
}

for col, mapping in category_maps.items():
    df[col] = df[col].cat.rename_categories(mapping)

```

In [11]: `df.tail()`

Out[11]:

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count	year	month_name	hour
10881	2012-12-19 19:00:00	winter	No	Yes	clear	15.58	19.695	50	26.0027	7	329	336	2012	December	19
10882	2012-12-19 20:00:00	winter	No	Yes	clear	14.76	17.425	57	15.0013	10	231	241	2012	December	20
10883	2012-12-19 21:00:00	winter	No	Yes	clear	13.94	15.910	61	15.0013	4	164	168	2012	December	21
10884	2012-12-19 22:00:00	winter	No	Yes	clear	13.94	17.425	61	6.0032	12	117	129	2012	December	22
10885	2012-12-19 23:00:00	winter	No	Yes	clear	13.12	16.665	66	8.9981	4	84	88	2012	December	23

Checking null values

In [12]: `df.isna().sum()`

Out[12]:

```

datetime      0
season        0
holiday       0
workingday    0
weather       0
temp         0
atemp        0
humidity      0
windspeed    0
casual        0
registered    0
count         0
year         0
month_name    0
hour         0
dtype: int64

```

As we can see there are no null values. Hence no data cleaning is required.

Univariate analysis

Numerical data

Now I will do Univariate analysis on the target variable that is count to check the distribution of the data. This distribution information will be crucial to apply the necessary statistical test.

In [13]:

```

fig, axes = plt.subplots(2,4, figsize=(25,10))
fig.suptitle("Numerical Univariate analysis", fontsize=18, weight="bold")

columns = ["count", "windspeed", "humidity", "temp"]
titles = ["Hourly Bike Rentals", "Hourly Windspeed", "Hourly Humidity", "Hourly Temperature"]

for i, (col, title) in enumerate(zip(columns, titles)):
    sns.histplot(
        data=df,
        x=col,
        kde=True,
        ax=axes[0,i]
    )

    axes[0, i].spines[["top", "right"]].set_visible(False)
    axes[0, i].set_title(title, weight="bold")
    axes[0, i].set_ylabel(None)

    if col == "count" or col == "windspeed" or col == "humidity" or col == "temp":
        axes[0, 0].set_xlabel("Bike Rented", weight="bold")
        axes[0, 1].set_xlabel("Windspeed", weight="bold")
        axes[0, 2].set_xlabel("Humidity", weight="bold")
        axes[0, 3].set_xlabel("Temperature", weight="bold")

    sns.boxplot(
        data=df,
        x=col,
        ax=axes[1,i]
    )

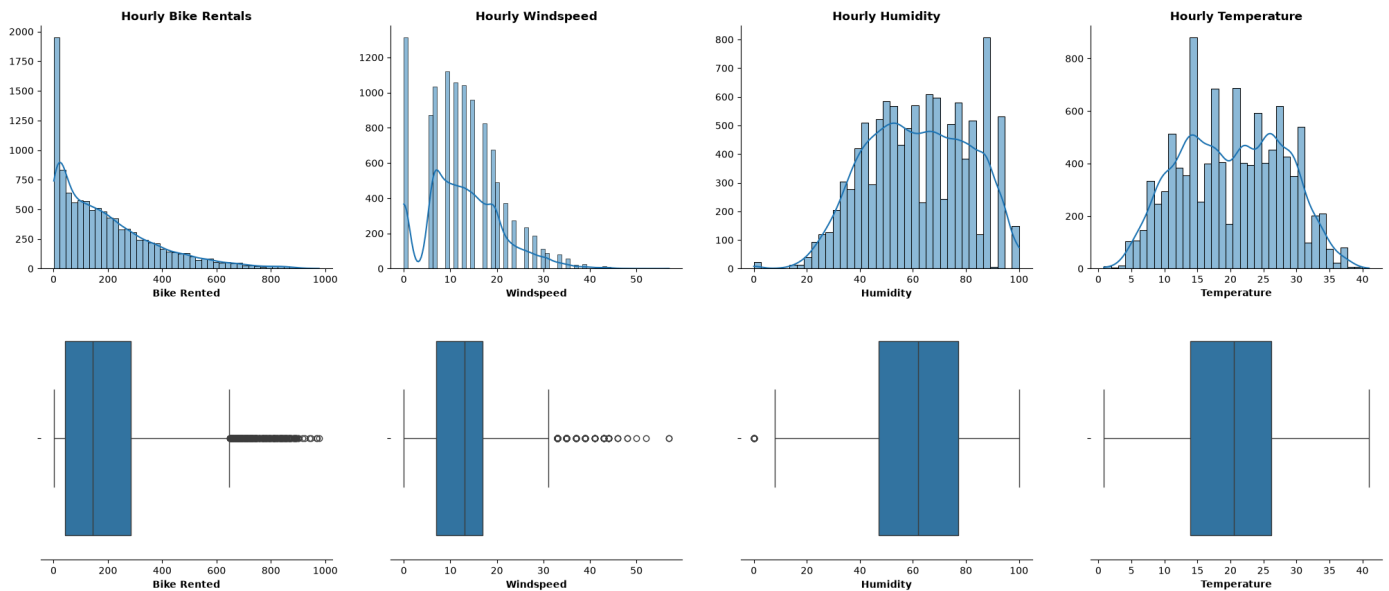
    axes[1, i].spines[["top", "right", "left"]].set_visible(False)

    if col == "count" or col == "windspeed" or col == "humidity" or col == "temp":
        axes[1,0].set_xlabel("Bike Rented", weight="bold")
        axes[1,1].set_xlabel("Windspeed", weight="bold")
        axes[1,2].set_xlabel("Humidity", weight="bold")
        axes[1,3].set_xlabel("Temperature", weight="bold")

plt.show()

```

Numerical Univariate analysis



```
In [14]: num_cols = ["count", "windspeed", "humidity", "temp"]

summary = df[num_cols].describe()

Q1 = df[num_cols].quantile(0.25)
Q3 = df[num_cols].quantile(0.75)

IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = ((df[num_cols] < lower_bound) |
            (df[num_cols] > upper_bound)).sum()

summary.loc["IQR"] = IQR
summary.loc["Lower Bound"] = lower_bound
summary.loc["Upper Bound"] = upper_bound
summary.loc["Outliers"] = outliers

summary
```

Out[14]:

	count	windspeed	humidity	temp
count	10886.000000	10886.000000	10886.000000	10886.000000
mean	191.574132	12.799395	61.886460	20.23086
std	181.144454	8.164537	19.245033	7.79159
min	1.000000	0.000000	0.000000	0.82000
25%	42.000000	7.001500	47.000000	13.94000
50%	145.000000	12.998000	62.000000	20.50000
75%	284.000000	16.997900	77.000000	26.24000
max	977.000000	56.996900	100.000000	41.00000
IQR	242.000000	9.996400	30.000000	12.30000
Lower Bound	-321.000000	-7.993100	2.000000	-4.51000
Upper Bound	647.000000	31.992500	122.000000	44.69000
Outliers	300.000000	227.000000	22.000000	0.00000

Count

- The count variable is positively (right) skewed, with the central 50% of observations lying between 42 and 284 rentals, based on Q1 and Q3.
- Most hourly observations have relatively low rental counts, while only a smaller number of hours experience very high rental demand, resulting in a long right tail.
- The IQR method identifies 300 observations above the upper bound of 647 rentals, indicating that these hours experienced unusually high rental counts relative to the central distribution.

Windspeed

- The windspeed variable is positively (right) skewed, with the central 50% of observations lying between 7 and 17, and a median of 13.
- Most hourly observations have relatively low windspeed values, while only a smaller number of hours experience very high windspeeds, reaching up to 57.
- The IQR method identifies 227 observations above the upper bound of approximately 32, indicating unusually high windspeed values compared with the central distribution.

Humidity

- The humidity variable is negatively (left) skewed, with the central 50% of observations lying between 47 and 77, and a median of 62.
- Most hourly observations occur at relatively higher humidity levels, while fewer observations occur at lower humidity levels.
- The IQR method identifies observations below the lower bound of 2 and 22 as potential outliers, indicating unusually low humidity values.

Temp

- The temp distribution appears multimodal, with multiple peaks rather than a single dominant peak. The mean and median are relatively close, and the central 50% of observations lie approximately between 14 and 26.
- The majority of observations are concentrated around moderate temperature values, while both very low and very high temperatures occur less frequently.

Categorical data

In [104...

```
fig, axes = plt.subplots(2, 2, figsize=(13, 10))
fig.suptitle("Categorical Variables - Univariate Analysis", y=1.02)

plot_config = [
    ("season", "Season Count", "Season"),
    ("weather", "Weather Count", "Weather"),
    ("holiday", "Holiday Count", None),
    ("workingday", "Working Day Count", None)
]

for ax, (column, title, xlabel) in zip(axes.flat, plot_config):

    sns.countplot(
        data=df,
        x=column,
        ax=ax
    )

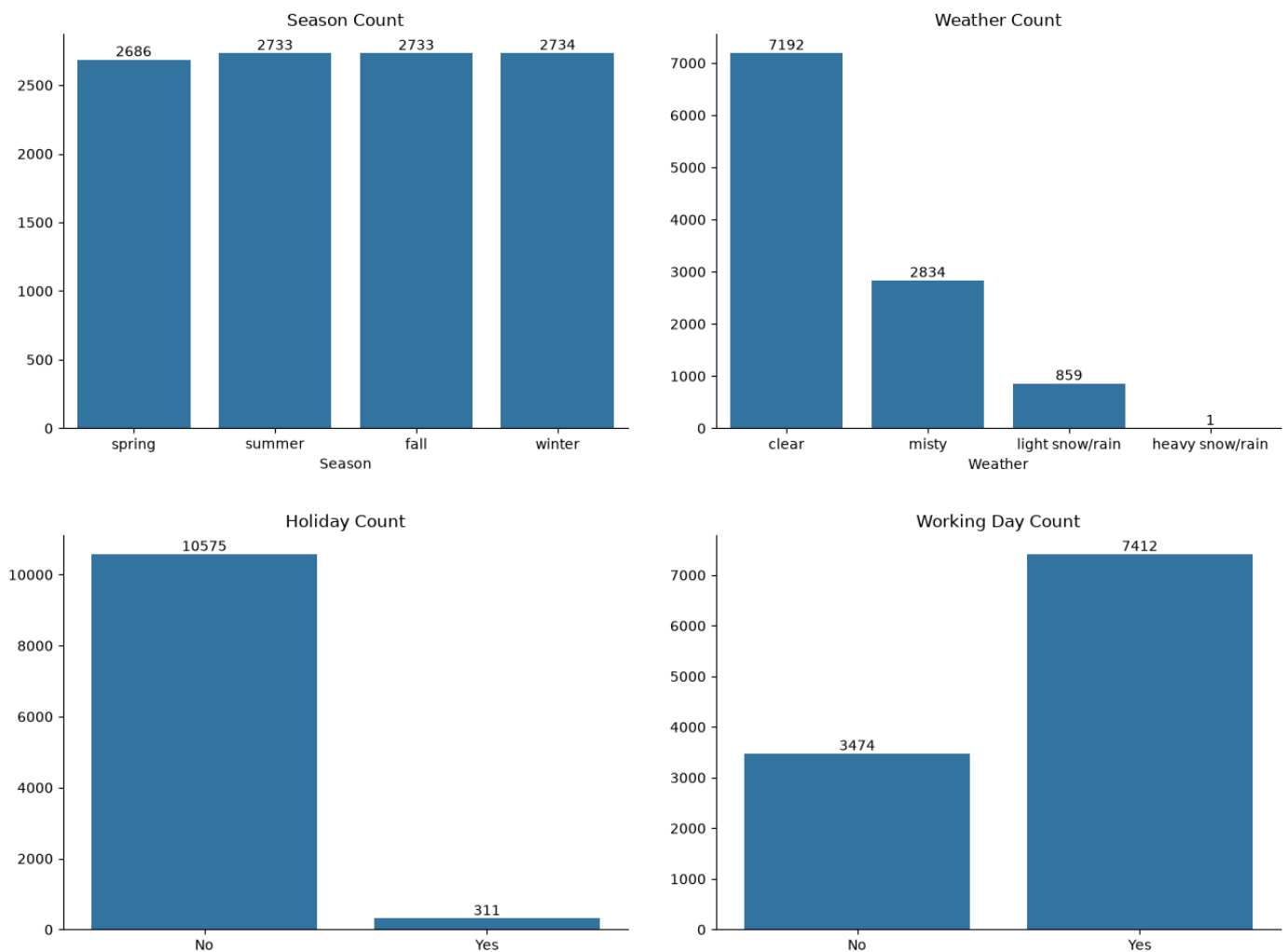
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.set_ylabel(None)

    ax.spines[["top", "right"]].set_visible(False)

    for container in ax.containers:
        ax.bar_label(container)

plt.tight_layout(h_pad=3.0, w_pad=3.0)
plt.show()
```

Categorical Variables - Univariate Analysis



Season

- The number of observations is relatively evenly distributed across seasons, with Spring having the lowest number of observations (2,686) and Winter having the highest (2,734).

Weather

- Weather category 1 (Clear/Partly Cloudy) has the highest number of observations (7,192), while Weather category 4 has only one observation, making it an extremely rare category in the dataset.

Holiday

- Non-holiday days account for the vast majority of observations (10,575), while only 311 observations correspond to holidays.

Workday

- Working days account for a substantially larger number of observations (7,412) compared with non-working days (3,474).

Bivariate analysis

Lets explore relationship between 2 variables now.

Numerical vs Categorical

- Season vs Count
- Weather vs Count
- Workingday vs Count

In [103...

```
fig, axes = plt.subplots(2, 3, figsize=(19, 11))
fig.suptitle("Numerical-Categorical Analysis", fontsize=16, y=1.02)

columns = ["season", "weather", "workingday"]
titles = ["Season vs Count", "Weather vs Count", "Working Day vs Count"]

for i, (col, title) in enumerate(zip(columns, titles)):

    sns.barplot(
        data=df,
        x=col,
        y="count",
        ax=axes[0, i]
    )

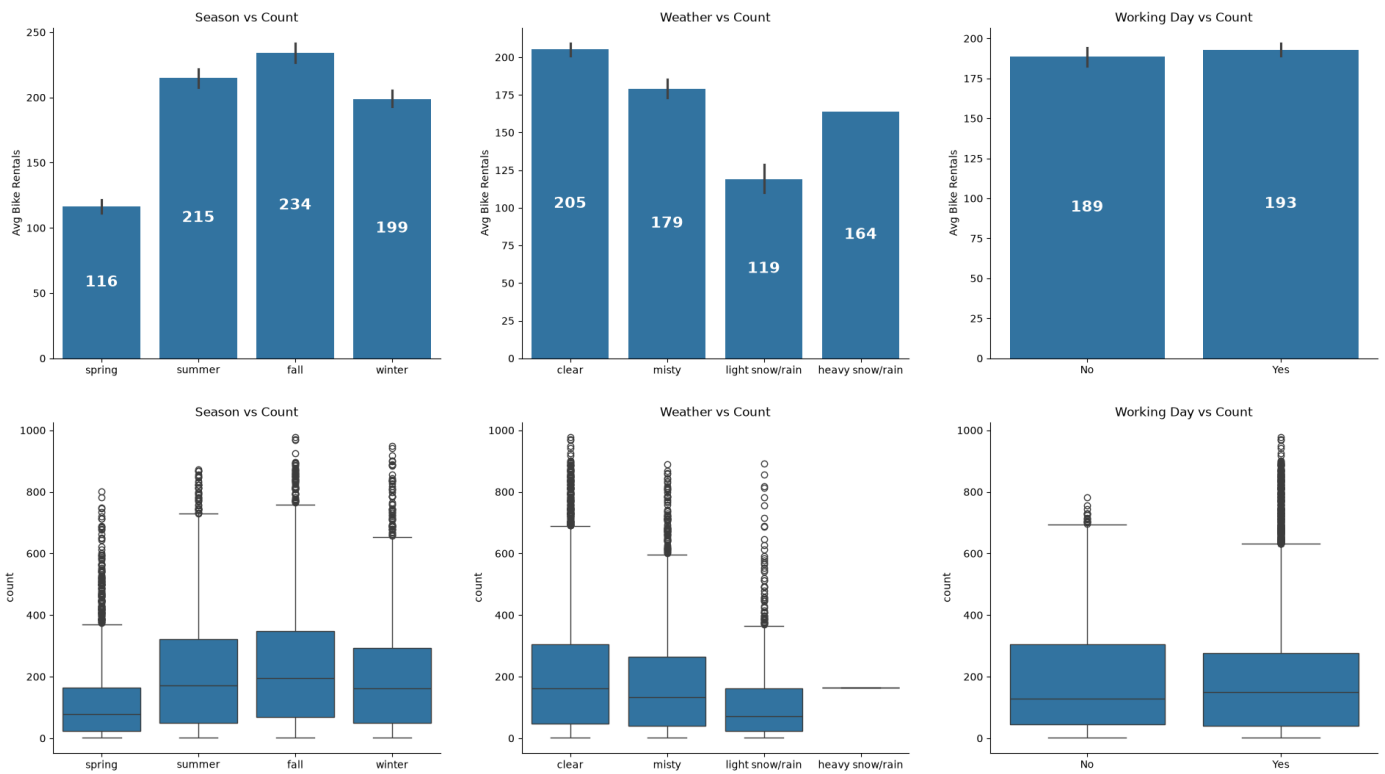
    for container in axes[0, i].containers:
        axes[0, i].bar_label(container, fmt="%0f", label_type="center", fontsize=15, weight="bold", color="white")

    sns.boxplot(
        data=df,
        x=col,
        y="count",
        ax=axes[1, i]
    )

    axes[0, i].spines[["top", "right"]].set_visible(False)
    axes[0, i].set_title(title)
    axes[0, i].set_ylabel("Avg Bike Rentals", xlabel=None)
    axes[1, i].spines[["top", "right"]].set_visible(False)
    axes[1, i].set_xlabel(None)
    axes[1, i].set_title(title)

plt.tight_layout(h_pad=3.0, w_pad=3.0)
plt.show()
```

Numerical-Categorical Analysis



Here are few observations:

Seasons

- Fall has the highest average hourly rental count (234), while Spring has the lowest (116), suggesting that average rental demand varies considerably across seasons.
- Spring has the lowest median rental count among the seasons, while several high-demand observations appear as potential outliers.
- Fall has the highest median rental count and the widest IQR among the four seasons, indicating greater variability in hourly rental demand.

Weather

- Weather category 1 has the highest average rental count (205), while weather category 3 has the lowest (119). This suggests that rental demand may vary across weather conditions.
- Weather category 3 has the lowest median rental count and shows a substantial number of high-demand observations beyond the upper whisker, indicating considerable variability within this category.
- Weather category 4 has a higher average rental count (164) than category 3 (119), despite having substantially fewer observations. The boxplot for category 4 is compressed/collapsed because of its very small sample size, so the observed mean should be interpreted cautiously.

Working day

- The average hourly rental count is slightly higher on working days (193) compared with non-working days (189).
- The non-working day category has a wider IQR than the working-day category, indicating greater variability in the middle 50% of rental counts.
- The median rental counts for working and non-working days are relatively similar, indicating that the typical rental demand is comparable between the two categories.
- The two boxplots show substantial overlap, suggesting that the difference in rental demand between working and non-working days may be relatively small compared with the overall variability.

Categorical vs Categorical

- Season vs Weather
- Weather vs Working Day

```
In [17]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle("Categorical-Categorical Analysis")

plot_config = [
    ("season", "weather", "Season vs Weather"),
    ("weather", "workingday", "Weather vs Working Day")
]

for ax, (x_col, hue_col, title) in zip(axes, plot_config):

    sns.countplot(
        data=df,
        x=x_col,
        hue=hue_col,
        ax=ax
    )

    ax.set_title(title, weight="bold")
    ax.set_xlabel(x_col.capitalize(), weight="bold")
    ax.set_ylabel(None)
    ax.spines[["top", "right"]].set_visible(False)

    if x_col == "season":
```

```

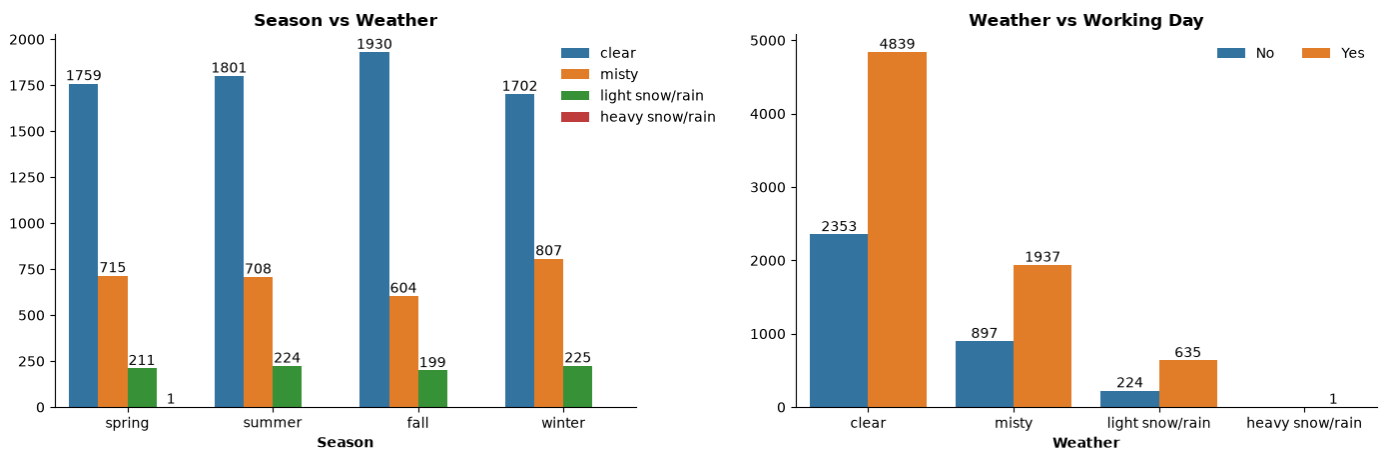
ax.legend(bbox_to_anchor=(0.85,1.0), frameon=False)
if x_col == "weather":
    ax.legend(frameon=False, ncol=2)

for container in ax.containers:
    ax.bar_label(container)

plt.tight_layout()
plt.show()

```

Categorical-Categorical Analysis



Season vs Weather

- Weather category 1 (Clear/Partly Cloudy) has the highest number of observations in every season, with the highest frequency occurring in Fall (1,930 observations).
- Weather category 2 is the second most frequently observed weather condition across all seasons, with the highest number of observations occurring in Winter (807).
- Category 3 Light rainy weather has almost same number of observations in all season but one observation higher in winter as compared to summer.
- Weather category 4 is extremely rare, with only one observation in the entire dataset, occurring during Spring.

Weather vs Workingday

- Working-day observations are higher across all weather categories, with Weather category 1 (Clear/Partly Cloudy) having the highest frequency.
- Non-working days also have observations across Weather categories 1, 2 and 3, but their frequencies are considerably lower than those of working days.
- Weather category 4 is extremely rare, with only one observation in the entire dataset, and it occurred on a working day.

Numerical vs Numerical

- Temp vs Count
- Humidity vs Count
- Windspeed vs Count

```

In [18]: fig, axes = plt.subplots(1, 3, figsize=(20,5))
fig.suptitle("Numerical vs Numerical analysis", y = 1.05, fontsize=15, weight="bold")

plot_config = [
    ("temp", "count", "Temp vs Bikes Rented", "Temperature"),
    ("windspeed", "count", "Windspeed vs Bikes Rented", "Windspeed"),
    ("humidity", "count", "Humidity vs Bikes Rented", "Humidity")
]

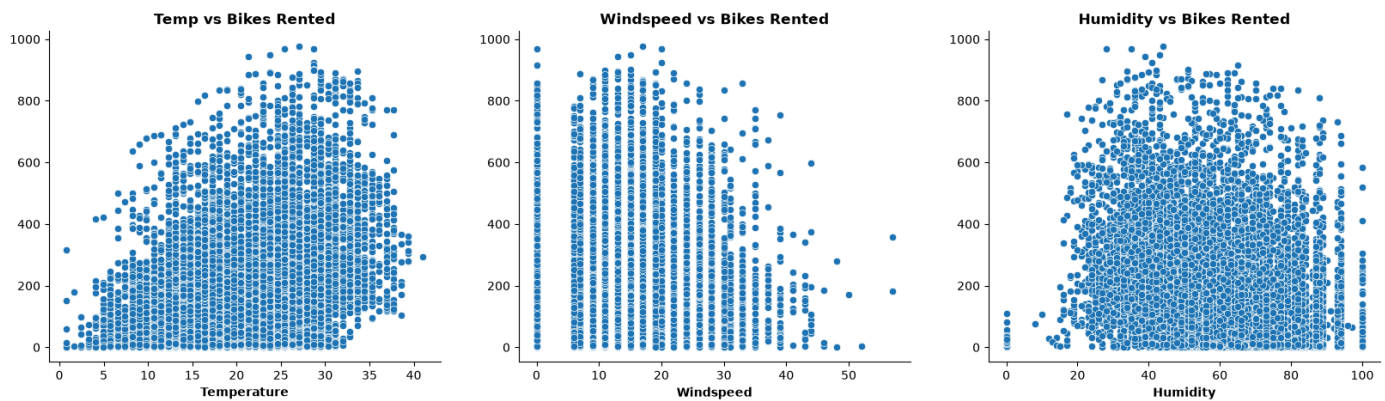
for ax, (x_col, y_col, title, x_label) in zip(axes, plot_config):
    sns.scatterplot(
        data = df,
        x = x_col,
        y = y_col,
        ax=ax
    )

    ax.set_title(title, weight="bold")
    ax.set_ylabel(None)
    ax.spines[["top", "right"]].set_visible(False)
    ax.set_xlabel(x_label.capitalize(), fontsize=10, weight="bold")

plt.show()

```


Numerical vs Numerical analysis



Temp vs Count

- Rental counts tend to increase as temperature rises from lower levels and reach higher values around 20-30°C. Beyond this range, rental counts generally tend to decline, suggesting that extremely high temperatures may be associated with lower rental demand.

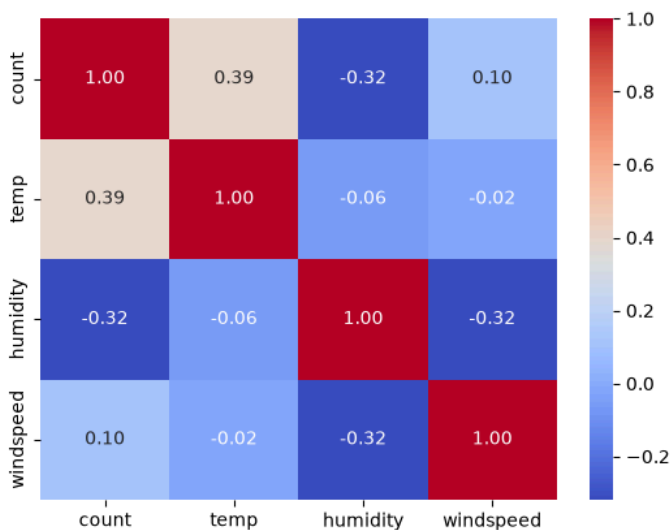
Windspeed vs Count

- Higher rental counts are more concentrated at lower to moderate windspeeds, while rental counts generally become lower as windspeed increases beyond approximately 20-25.

Humidity vs Count

- Higher rental counts are more frequently observed at moderate humidity levels, while very high humidity levels generally show fewer high rental counts.

```
In [19]: sns.heatmap(df[["count", "temp", "humidity", "windspeed"]].corr(), cmap="coolwarm", annot=True, fmt=".2f")
plt.show()
```



Temp vs Count

- Temp and count show a moderate positive correlation ($r \approx 0.39$), suggesting that bike rentals generally tend to increase as temperature increases. However, the scatterplot shows considerable dispersion, indicating that temperature alone does not explain rental demand.

Windspeed vs Count

- Windspeed and count have a very weak positive linear correlation ($r = 0.10$). However, the scatterplot does not show a clear strong linear pattern, indicating that windspeed has a weak linear association with rental count.

Humidity vs Count

- Humidity and count have a moderate negative Pearson correlation ($r = -0.32$). However, the scatterplot does not show a clear linear pattern, suggesting that the relationship may not be purely linear.

Hypothesis Testing

Does Workingday has effect on number of electric cycles rented

```
In [20]: df.groupby("workingday")["count"].describe()
```

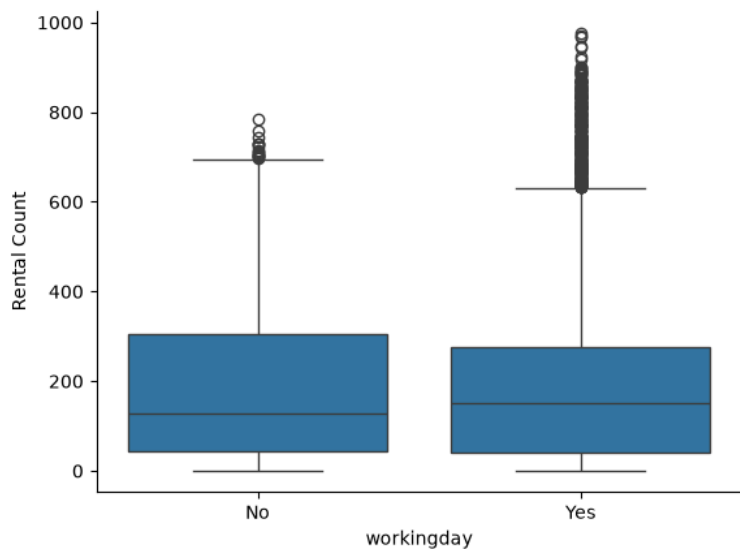
```
Out[20]:
```

	count	mean	std	min	25%	50%	75%	max
workingday								
No	3474.0	188.506621	173.724015	1.0	44.0	128.0	304.0	783.0
Yes	7412.0	193.011873	184.513659	1.0	41.0	151.0	277.0	977.0

```
In [102]: fig, ax = plt.subplots()

sns.boxplot(df, x="workingday", y="count")
ax.spines[["top", "right"]].set_visible(False)
ax.set(ylabel="Rental Count")

plt.show()
```



```
In [22]: # Checking Normality

working = df.loc[df["workingday"] == "Yes", "count"]
non_working = df.loc[df["workingday"] == "No", "count"]

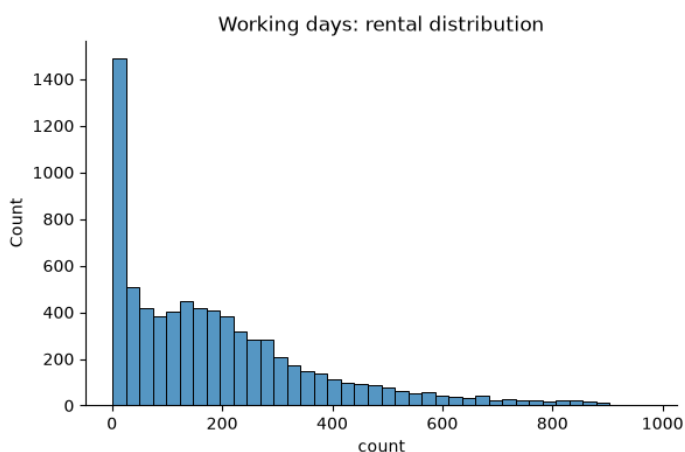
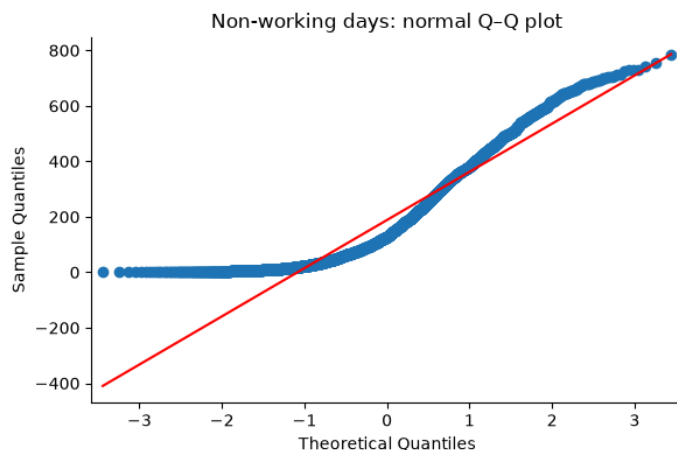
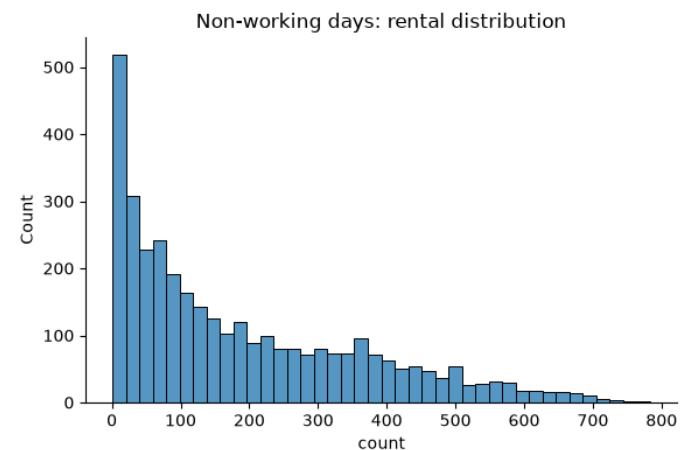
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

for row, (label, values) in enumerate([("Non-working days", non_working), ("Working days", working)]):
    sns.histplot(values, bins=40, ax=axes[row, 0])
    axes[row, 0].set_title(f"{label}: rental distribution")

    sm.qqplot(values, line="s", ax=axes[row, 1])
    axes[row, 1].set_title(f"{label}: normal Q-Q plot")

    axes[row, 0].spines[["top", "right"]].set_visible(False)
    axes[row, 1].spines[["top", "right"]].set_visible(False)

plt.tight_layout()
plt.show()
```



Normality check:

- The rental-count distributions for both working and non-working days are positively skewed. The Q-Q plots also show substantial deviation from the normal reference line, indicating that the data are not normally distributed.

```
In [23]: # Checking Variances

H0 = "The variances of both groups are equal"
Ha = "The variances of both groups are different"

levene_stat, p_value = stats.levene(working, non_working)

print(f"Levene Statistic: {levene_stat:.4f}")
print(f"P-Value: {p_value:.4f}")

alpha = 0.05

if p_value < alpha:
    print(f"Reject H0: {Ha}")
else:
    print(f"Fail to reject H0: {H0}")
```

Levene Statistic: 0.0050
P-Value: 0.9438
Fail to reject H0: The variances of both groups are equal

There is insufficient evidence to conclude that the variances of rental counts differ between working and non-working days. Therefore, the equal-variance assumption can be considered satisfied for the independent two-sample t-test.

```
In [24]: # Testing with 2-Sample T-test

H0 = "Mean rental count is equal on working and non-working days"
Ha = "Mean rental count differs between working and non-working days"

t_stat, p_value = stats.ttest_ind(working, non_working)

print(f"T-Stat: {t_stat:.4f}")
print(f"P-Value: {p_value:.4f}")

alpha = 0.05

if p_value < alpha:
    print(f"Reject H0: {Ha}")
else:
    print(f"Fail to reject H0: {H0}")
```

T-Stat: 1.2096
P-Value: 0.2264
Fail to reject H0: Mean rental count is equal on working and non-working days

At the 5% significance level, there is insufficient statistical evidence to conclude that the mean rental count differs between working and non-working days (t = 1.2096, p = 0.2264) .

Are number of cycles rented is similar or different in different weathers

```
In [25]: df.groupby("weather")["count"].describe()
```

```
Out[25]:
```

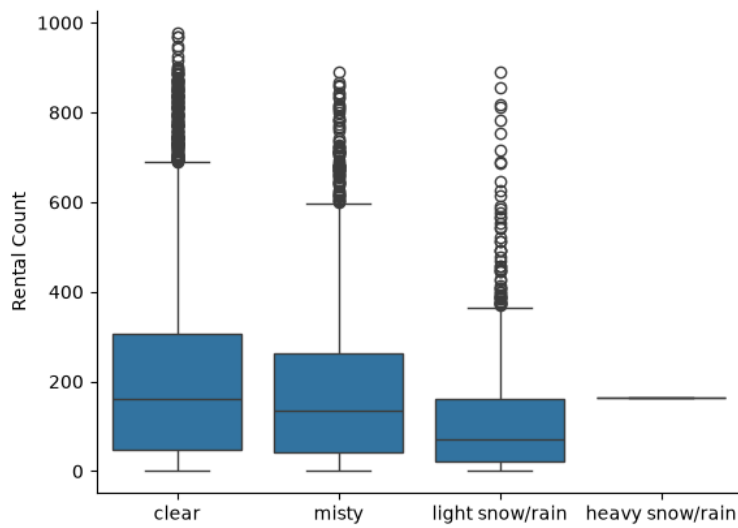
	count	mean	std	min	25%	50%	75%	max
weather								
clear	7192.0	205.236791	187.959566	1.0	48.0	161.0	305.0	977.0
misty	2834.0	178.955540	168.366413	1.0	41.0	134.0	264.0	890.0
light snow/rain	859.0	118.846333	138.581297	1.0	23.0	71.0	161.0	891.0
heavy snow/rain	1.0	164.000000	NaN	164.0	164.0	164.0	164.0	164.0

Weather category 4 (Heavy Snow/Rain) contains only one observation (n=1). Therefore, its standard deviation and variance cannot be estimated, and normality or homogeneity of variance cannot be meaningfully assessed for this category. Consequently, category 4 was excluded from the assumption checks, while its limited sample size is reported as a limitation of the analysis.

```
In [100... fig, ax = plt.subplots()

sns.boxplot(df, x="weather", y="count")
ax.spines[["top", "right"]].set_visible(False)
ax.set(ylabel="Rental Count", xlabel=None)

plt.show()
```



```
In [27]: clear = df.loc[(df["weather"] == "clear"), "count"]
misty = df.loc[(df["weather"] == "misty"), "count"]
light_snow_rain = df.loc[(df["weather"] == "light snow/rain"), "count"]
```

```
In [28]: fig, axes = plt.subplots(2, 3, figsize=(15, 8))

weather_groups = [
    ("Clear", clear),
    ("Misty", misty),
    ("Light Snow/Rain", light_snow_rain),
]

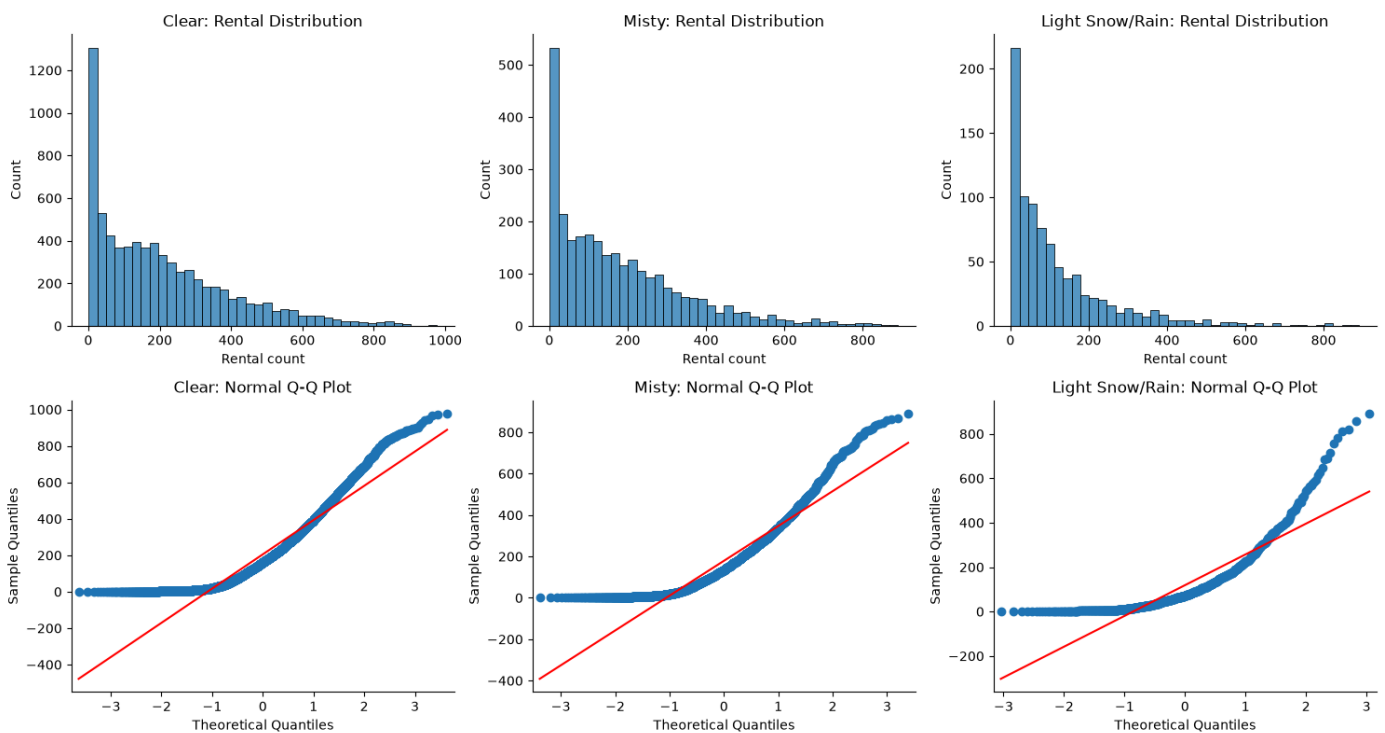
for col, (label, values) in enumerate(weather_groups):

    sns.histplot(values, bins=40, ax=axes[0, col])
    axes[0, col].set_title(f"{label}: Rental Distribution")
    axes[0, col].set_xlabel("Rental count")

    sm.qqplot(values, line="s", ax=axes[1, col])
    axes[1, col].set_title(f"{label}: Normal Q-Q Plot")

    for ax in axes[:, col]:
        ax.spines[["top", "right"]].set_visible(False)

plt.tight_layout()
plt.show()
```



Observations

- There is statistically significant evidence that the rental-count variances are not equal across the weather categories. Therefore, the homogeneity-of-variance assumption for standard one-way ANOVA is violated.

```
In [29]: # Checking Variances

H0 = "The variances of all weather groups are equal"
Ha = "At least one group's variance differs"

levene_stat, p_value = stats.levene(clear, misty, light_snow_rain)
```

```
print(f"Levene Statistic: {levevene_stat:.4f}")
print(f"P-Value: {p_value:.4e}")

alpha = 0.05

if p_value < alpha:
    print(f"Reject H0: {Ha}")
else:
    print(f"Fail to reject H0: {H0}")
```

Levene Statistic: 81.6757

P-Value: 6.1983e-36

Reject H0: At least one group's variance differs

- The homogeneity-of-variance assumption was violated (Levene's test: $p < 0.05$). Additionally, the Heavy Snow/Rain category contains only one observation, making its distribution and variance impossible to assess reliably. The required one-way ANOVA will still be performed, but its result should be interpreted cautiously.

In [30]: *# Checking with Welch's Anova (anova_oneway) since assumption of equal variance fail for One way Anova (f_oneway)*

```
H0 = "Mean rental count is equal across all three weather groups."
Ha = "At least one weather group has a different mean rental count."

welch_result, p_value = anova_oneway(
    data=[clear, misty, light_snow_rain],
    use_var="unequal"
)

print(f"Welch's Result: {welch_result:.4f}")
print(f"P Value: {p_value:.4e}")

alpha = 0.05

if p_value < alpha:
    print(f"Reject H0: {Ha}")
else:
    print(f"Fail to reject H0: {H0}")
```

Welch's Result: 140.9007

P Value: 1.2002e-58

Reject H0: At least one weather group has a different mean rental count.

In [31]: *H0 = "The rental-count distributions are the same across the weather groups."*
Ha = "At least one weather group's rental-count distribution differs."

```
h_stat, p_value = stats.kruskal(
    clear,
    misty,
    light_snow_rain
)

print(f"H-Statistic: {h_stat:.4f}")
print(f"P-Value: {p_value:.4e}")

alpha = 0.05

if p_value < alpha:
    print(f"Reject H0: {Ha}")
else:
    print(f"Fail to reject H0: {H0}")
```

H-Statistic: 204.9557

P-Value: 3.1221e-45

Reject H0: At least one weather group's rental-count distribution differs.

Both Welch's ANOVA and the Kruskal–Wallis test provide statistically significant evidence of differences in rental demand across the adequately represented weather categories. Welch's ANOVA indicates differences in mean rental counts, while Kruskal–Wallis provides supporting evidence that the rental-count distributions differ across weather conditions.

Conclusion

Rental counts differ significantly across weather conditions. Clear weather has the highest average rental count, followed by Misty weather and Light Snow/Rain. Both Welch's ANOVA and the Kruskal–Wallis test reject the null hypothesis at the 5% significance level, providing strong evidence that weather condition is associated with differences in rental demand. Heavy Snow/Rain was excluded from inferential testing because it contained only one observation and therefore could not provide a reliable estimate of group variability.

Are number of cycles rented is similar or different in different seasons

In [32]: `df.groupby("season")["count"].describe()`

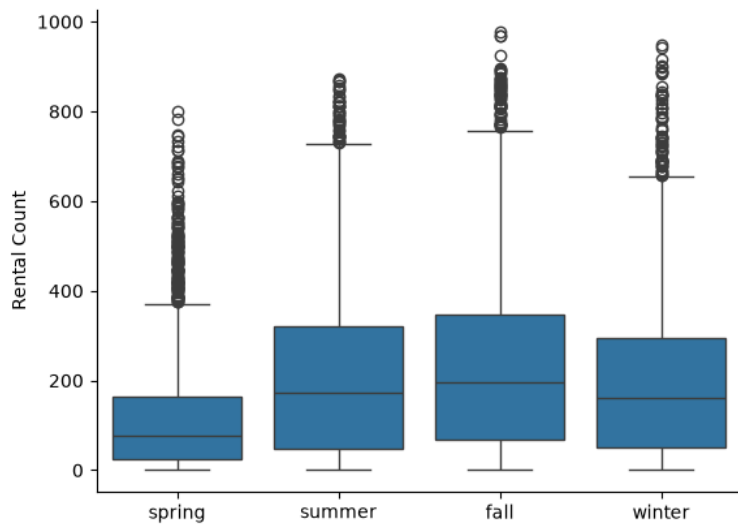
Out[32]:

	count	mean	std	min	25%	50%	75%	max
season								
spring	2686.0	116.343261	125.273974	1.0	24.0	78.0	164.0	801.0
summer	2733.0	215.251372	192.007843	1.0	49.0	172.0	321.0	873.0
fall	2733.0	234.417124	197.151001	1.0	68.0	195.0	347.0	977.0
winter	2734.0	198.988296	177.622409	1.0	51.0	161.0	294.0	948.0

In [97]: `fig, ax = plt.subplots()`
`sns.boxplot(df, x="season", y="count")`

```
ax.spines[["top", "right"].set_visible(False)
ax.set(ylabel="Rental Count", xlabel=None)

plt.show()
```



```
In [34]: spring = df.loc[df["season"] == "spring", "count"]
summer = df.loc[df["season"] == "summer", "count"]
fall = df.loc[df["season"] == "fall", "count"]
winter = df.loc[df["season"] == "winter", "count"]
```

```
In [35]: fig, axes = plt.subplots(2, 4, figsize=(20, 8))

season_groups = [
    ("Spring", spring),
    ("Summer", summer),
    ("Fall", fall),
    ("Winter", winter)
]

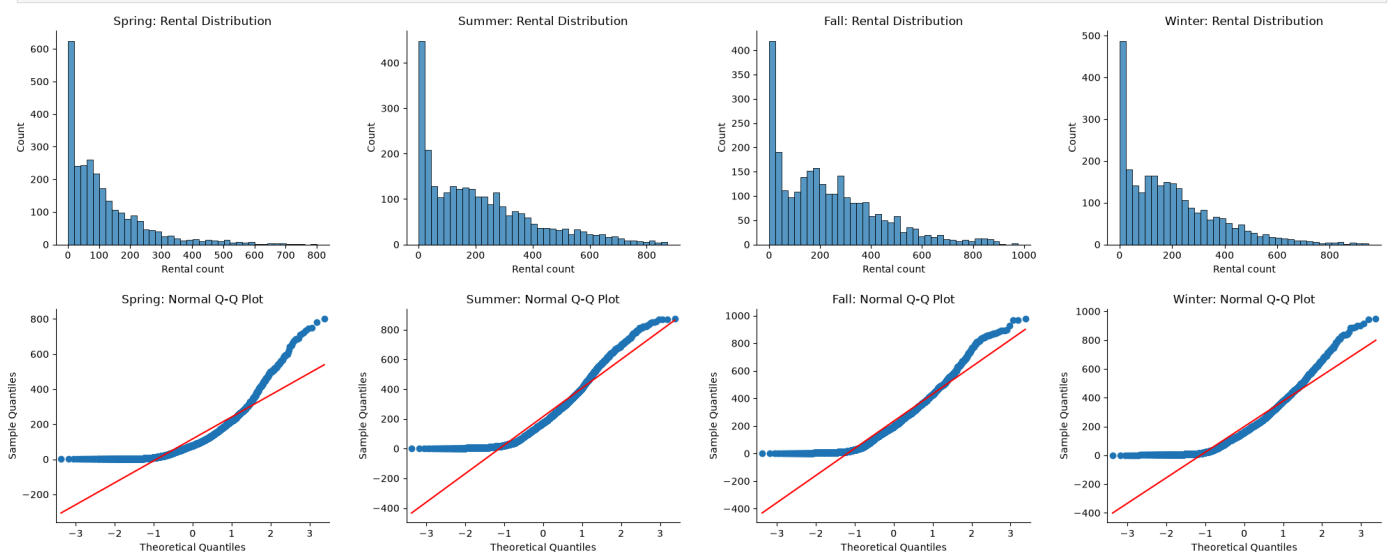
for col, (label, values) in enumerate(season_groups):

    sns.histplot(values, bins=40, ax=axes[0, col])
    axes[0, col].set_title(f"{label}: Rental Distribution")
    axes[0, col].set_xlabel("Rental count")

    sm.qqplot(values, line="s", ax=axes[1, col])
    axes[1, col].set_title(f"{label}: Normal Q-Q Plot")

    for ax in axes[:, col]:
        ax.spines[["top", "right"].set_visible(False)

plt.tight_layout(h_pad=2, w_pad=3)
plt.show()
```



```
In [36]: # Checking Variances

H0 = "The variances of all season groups are equal"
Ha = "At least one group's variance differs"

levene_stat, p_value = stats.levene(spring, summer, fall, winter)

print(f"Levene Statistic: {levene_stat:.4f}")
print(f"P-Value: {p_value:.4e}")

alpha = 0.05

if p_value < alpha:
```

```

    print(f"Reject H0: {Ha}")
else:
    print(f"Fail to reject H0: {H0}")

```

Levene Statistic: 187.7707

P-Value: 1.0147e-118

Reject H0: At least one group's variance differs

In [37]: *# Checking with Welch's Anova (anova_oneway) since assumption of equal variance fail for One way Anova (f_oneway)*

```

H0 = "Mean rental count is equal across all seasons."
Ha = "At least one season has a different mean rental count."

welch_result, p_value = anova_oneway(
    data=[spring, summer, fall, winter],
    use_var="unequal"
)

print(f"Welch's Result: {welch_result:.4f}")
print(f"P Value: {p_value:.4e}")

alpha = 0.05

if p_value < alpha:
    print(f"Reject H0: {Ha}")
else:
    print(f"Fail to reject H0: {H0}")

```

Welch's Result: 335.0889

P Value: 4.8450e-201

Reject H0: At least one season has a different mean rental count.

In [38]: *H0 = "The rental-count distributions are the same across the seasons."*

Ha = "At least one season's rental-count distribution differs."

```

h_stat, p_value = stats.kruskal(
    spring,
    summer,
    fall,
    winter
)

print(f"H-Statistic: {h_stat:.4f}")
print(f"P-Value: {p_value:.4e}")

alpha = 0.05

if p_value < alpha:
    print(f"Reject H0: {Ha}")
else:
    print(f"Fail to reject H0: {H0}")

```

H-Statistic: 699.6669

P-Value: 2.4790e-151

Reject H0: At least one season's rental-count distribution differs.

Rental demand differs significantly across seasons. Welch's ANOVA provides strong evidence that at least one seasonal mean rental count differs, while the Kruskal-Wallis test also shows that the rental-count distributions differ across seasons

Conclusion

- Rental demand varies significantly across seasons. Both Welch's ANOVA and the Kruskal-Wallis test reject the null hypothesis at the 5% significance level, indicating statistically significant seasonal differences in rental demand. Descriptively, Fall shows the highest average rental count, followed by Summer, Winter, and Spring.

If Weather is dependent on Season

In [39]: *H0 = "Weather and Season are independent"*

Ha = "Weather and Season are associated"

In [40]: *observed_df = pd.crosstab(
 df.loc[df["weather"] != "heavy snow/rain", "weather"],
 df.loc[df["weather"] != "heavy snow/rain", "season"]
)*

The Heavy Snow/Rain category contained only one observation. This resulted in expected frequencies below the acceptable level for the Chi-square approximation and made reliable comparison of this category impossible. Therefore, Heavy Snow/Rain was excluded from the inferential Chi-square analysis. The test was subsequently performed using Clear, Misty, and Light Snow/Rain weather categories only.

In [41]: *observed_df*

Out[41]:

	season	spring	summer	fall	winter
weather					
clear	1759	1801	1930	1702	
misty	715	708	604	807	
light snow/rain	211	224	199	225	

In [42]: *chi_stat, p_value, dof, exp_freq = stats.chi2_contingency(observed_df) # chi_stat, p_value, df, expected values*

```

print(f"Chi-stat: {chi_stat:.2f}")
print(f"P-value: {p_value:.2e}")

```

```
print(f"dof: {dof}")
print(f"exp_freq: {exp_freq}")
```

```
Chi-stat: 46.10
P-value: 2.83e-08
dof: 6
exp_freq: [[1774.04869086 1805.76352779 1805.76352779 1806.42425356]
 [ 699.06201194  711.55920992  711.55920992  711.81956821]
 [ 211.8892972   215.67726229  215.67726229  215.75617823]]
```

```
In [43]: alpha = 0.05

if p_value < alpha:
    print(f"Reject H0: {Ha}")
else:
    print(f"Fail to reject H0: {H0}")
```

Reject H0: Weather and Season are associated

Since the p-value is below the 0.05 significance level, the null hypothesis is rejected. There is statistically significant evidence of an association between season and weather condition among the three adequately represented weather categories.

Does Holiday has effect on number of electric cycles rented

```
In [44]: df.groupby("holiday")["count"].sum().reset_index()
```

```
Out[44]:
```

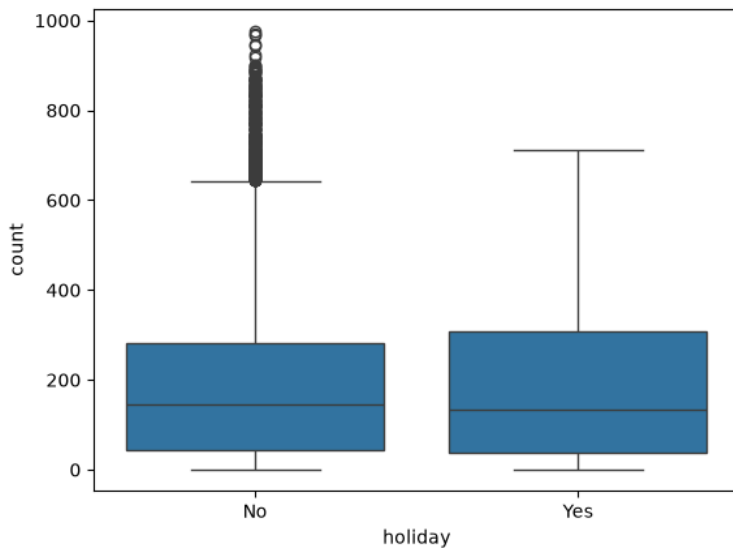
	holiday	count
0	No	2027668
1	Yes	57808

```
In [45]: df.groupby("holiday")["count"].describe()
```

```
Out[45]:
```

	count	mean	std	min	25%	50%	75%	max
holiday								
No	10575.0	191.741655	181.513131	1.0	43.0	145.0	283.0	977.0
Yes	311.0	185.877814	168.300531	1.0	38.5	133.0	308.0	712.0

```
In [60]: sns.boxplot(df, x="holiday", y="count")
plt.show()
```



```
In [57]: holiday = df.loc[df["holiday"] == "Yes", "count"]
nonholiday = df.loc[df["holiday"] == "No", "count"]
```

```
In [105... fig, axes = plt.subplots(2, 2, figsize=(13, 10))

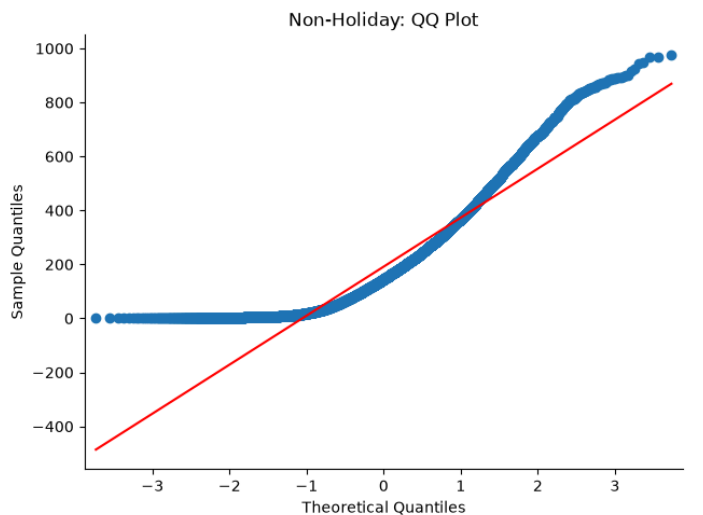
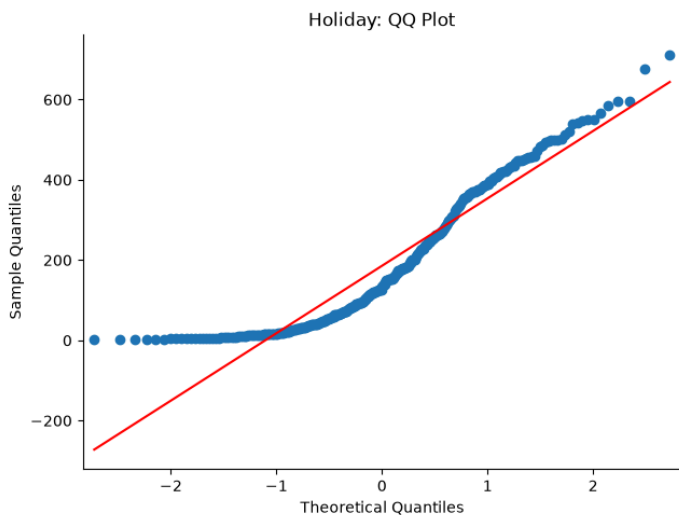
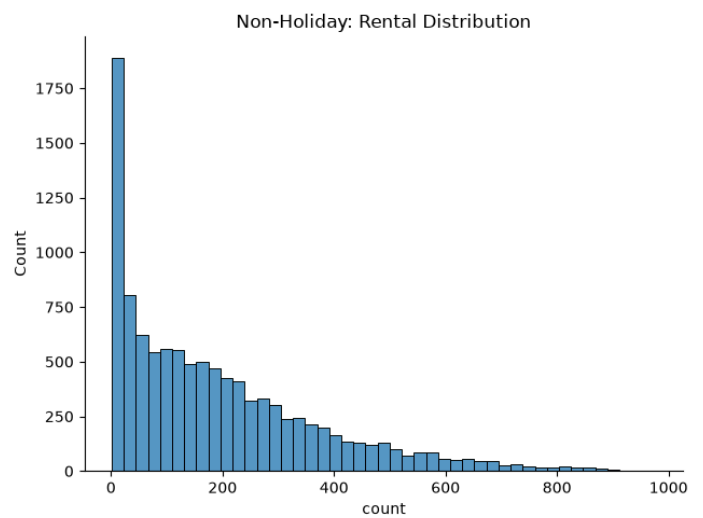
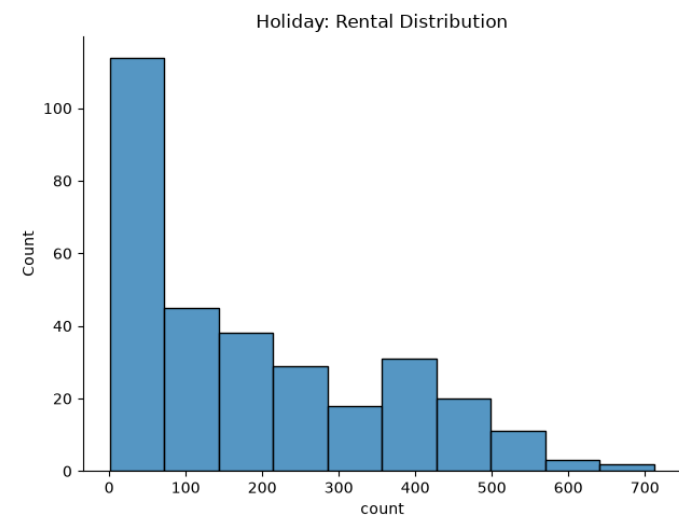
holiday_group = [
    ("Holiday", holiday),
    ("Non-Holiday", nonholiday)
]

for rows, (labels, values) in enumerate(holiday_group):

    sns.histplot(values, ax=axes[0, rows])
    axes[0, rows].set_title(f"{labels}: Rental Distribution")
    axes[0, rows].spines[["top", "right"]].set_visible(False)

    sm.qqplot(values, line="s", ax=axes[1, rows])
    axes[1, rows].set_title(f"{labels}: QQ Plot")
    axes[1, rows].spines[["top", "right"]].set_visible(False)

plt.tight_layout(h_pad=2.5)
plt.show()
```

```
In [113]: # Checking Variances

H0 = "The variances of holiday and non-holiday are equal"
Ha = "The variances of holiday and non-holiday are different"

levene_stat, p_value = stats.levene(holiday, nonholiday)

print(f"Levene Statistic: {levene_stat:.4e}")
print(f"P-Value: {p_value:.4f}")

alpha = 0.05

if p_value < alpha:
    print(f"Reject H0: {Ha}")
else:
    print(f"Fail to reject H0: {H0}")
```

Levene Statistic: 1.2223e-06
P-Value: 0.9991
Fail to reject H0: The variances of holiday and non-holiday are equal

```
In [115]: # Testing with 2-Sample T-test

H0 = "Mean rental count is equal on holidays and nonholidays"
Ha = "Mean rental count differs between holiday and nonholidays"

t_stat, p_value = stats.ttest_ind(holiday, nonholiday)

print(f"T-Stat: {t_stat:.4f}")
print(f"P-Value: {p_value:.4f}")

alpha = 0.05

if p_value < alpha:
    print(f"Reject H0: {Ha}")
else:
    print(f"Fail to reject H0: {H0}")
```

T-Stat: -0.5626
P-Value: 0.5737
Fail to reject H0: Mean rental count is equal on holidays and nonholidays

Additonal EDA

Multivariate analysis

```
In [48]: fig, ax = plt.subplots(figsize=(15,6))
```

```

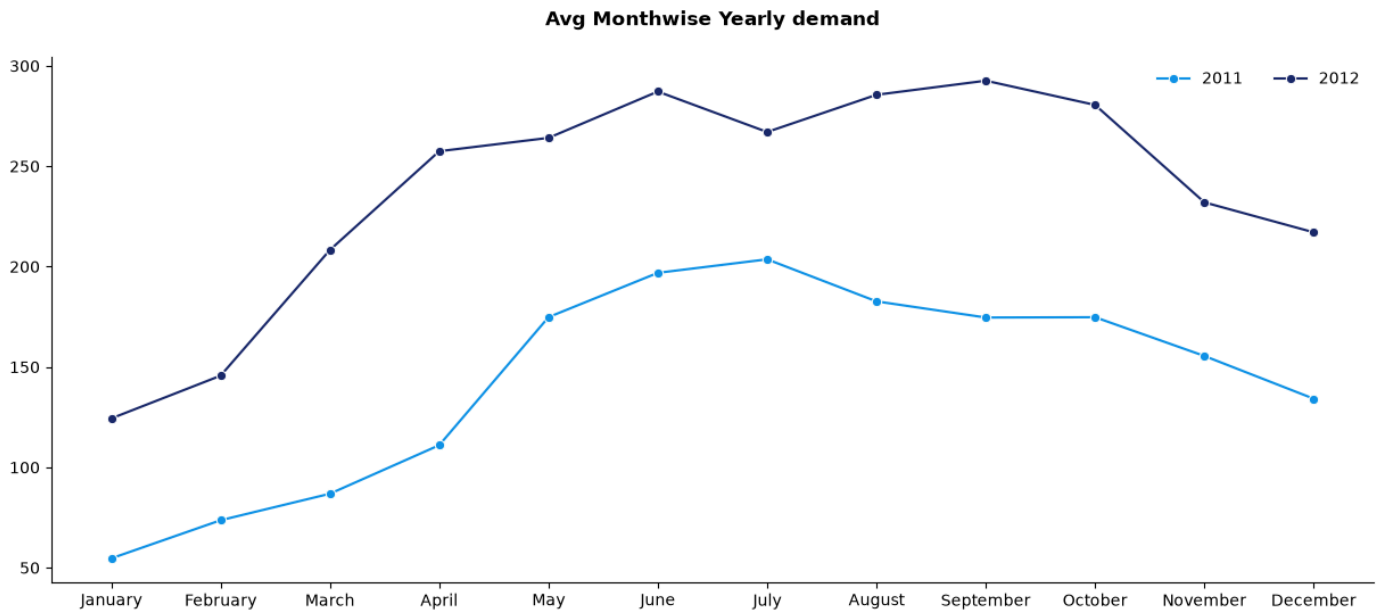
year_palette = ["#0F95E9", "#1B2A6D"]

sns.lineplot(df.groupby(["year", "month_name"])["count"].mean().reset_index(),
             x="month_name", y="count", hue="year", marker="o", palette=year_palette)

ax.spines[["top", "right"]].set_visible(False)
ax.legend(ncol=2, frameon=False)
ax.set_title("Avg Monthwise Yearly demand", pad=20, weight="bold")
ax.set_xlabel(None)
ax.set_ylabel(None)

plt.show()

```



In [151]:

```

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

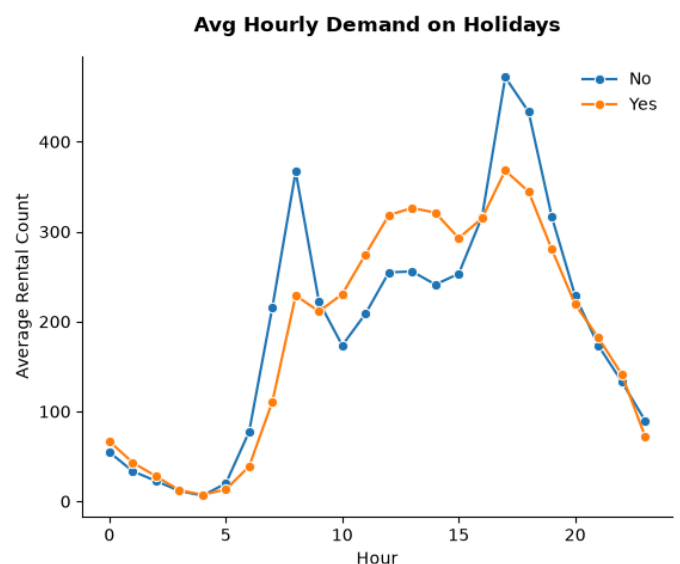
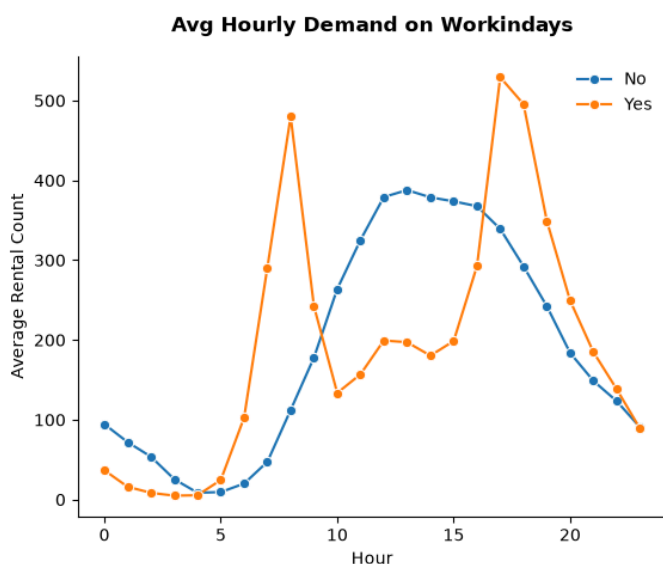
avg_workingday_hourly_rentals = df.groupby(["workingday", "hour"], as_index=False)["count"].mean()
avg_holiday_hourly_rentals = df.groupby(["holiday", "hour"], as_index=False)["count"].mean()

plot = [
    ("Avg Hourly Demand on Workindays", "hour", "count", "workingday", avg_workingday_hourly_rentals),
    ("Avg Hourly Demand on Holidays", "hour", "count", "holiday", avg_holiday_hourly_rentals)
]

for ax, (labels, x_val, y_val, hue, data) in zip(axes, plot):
    sns.lineplot(
        data,
        x=x_val,
        y=y_val,
        hue=hue,
        marker="o",
        ax=ax
    )
    ax.set_title(labels, weight="bold", pad=15)
    ax.set_xlabel("Hour")
    ax.set_ylabel("Average Rental Count")
    ax.spines[["top", "right"]].set_visible(False)
    ax.legend(frameon=False)

plt.show()

```



In [50]:

```

user_group = df.groupby("month_name", as_index=True)[["casual", "registered"]].sum().reset_index()

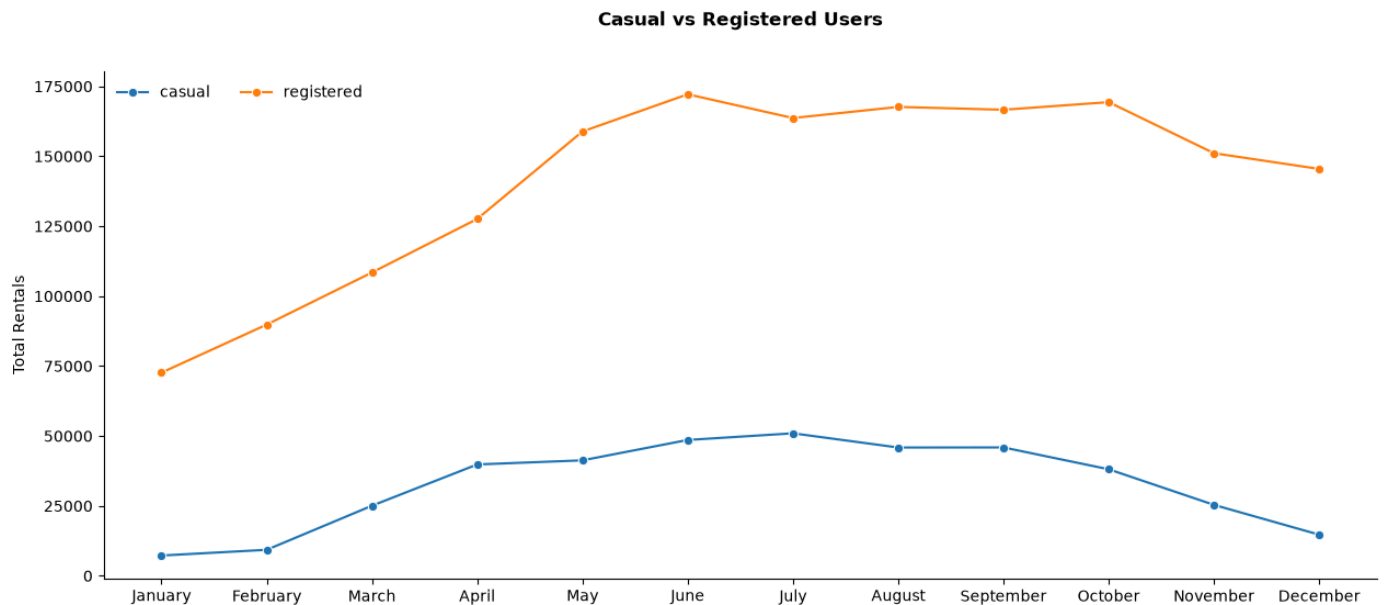
```

```
In [51]: user_trend = pd.melt(user_group, id_vars=["month_name"], value_vars=["casual","registered"],
                             var_name="user_category", value_name="sum of users")
```

```
In [52]: fig, ax = plt.subplots(figsize=(15, 6))

sns.lineplot(
    user_trend,
    x="month_name",
    y="sum of users",
    hue="user_category",
    marker="o"
)

ax.legend(ncol=2, frameon=False)
ax.spines[["top","right"]].set_visible(False)
ax.set_title("Casual vs Registered Users", pad=30, weight="bold")
ax.set_ylabel("Total Rentals")
ax.set_xlabel(None)
plt.show()
```

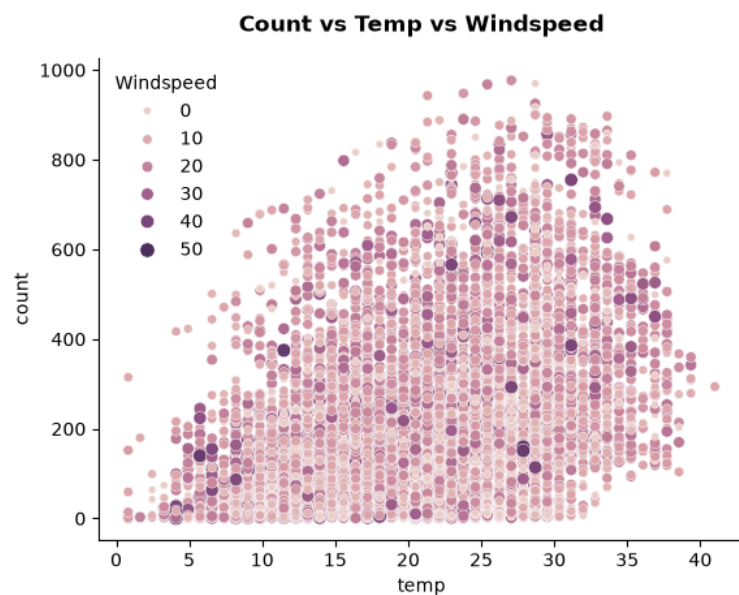


Numerical vs Numerical vs Numerical

```
In [53]: fig ,ax = plt.subplots()

sns.scatterplot(
    data=df,
    x="temp",
    y="count",
    size="windspeed",
    hue="windspeed"
)

ax.set_title("Count vs Temp vs Windspeed", pad=15, weight="bold")
ax.spines[["top", "right"]].set_visible(False)
ax.legend(title="Windspeed", frameon=False)
plt.show()
```



- Higher rental counts are predominantly observed between 20-30°C, particularly at low-to-moderate windspeeds (approximately 0-20).
- At lower temperatures, observations with higher windspeeds generally correspond to lower rental counts.
- Most observations are concentrated at windspeeds between approximately 0-30, with relatively fewer observations at higher windspeeds.

- Extremely high rental counts are concentrated around moderate temperatures, while very low and very high temperatures show fewer observations with exceptionally high demand.
- Although high rental counts are more frequently observed at lower-to-moderate windspeeds, some relatively high rental counts also occur at higher windspeeds, suggesting that windspeed alone does not determine rental demand.

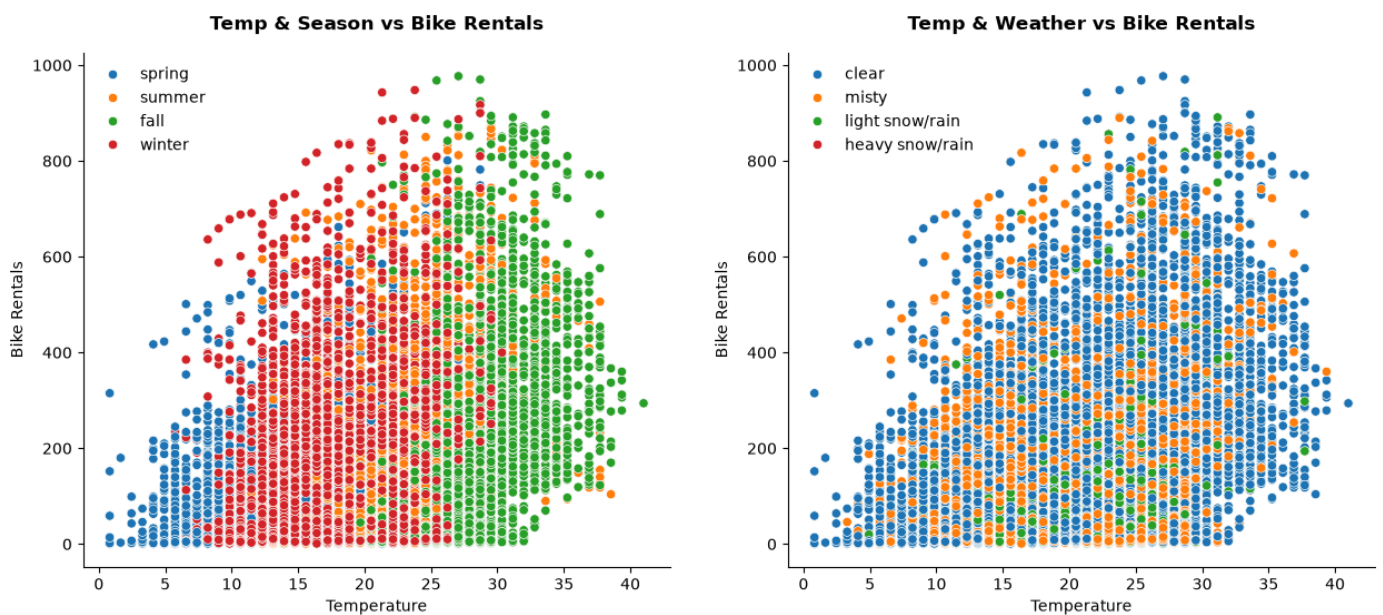
Numerical vs Numerical vs Categorical

```
In [54]: fig, axes = plt.subplots(1, 2, figsize=(15,6))

plot_config = [
    ("temp", "count", "season", "Temp & Season vs Bike Rentals"),
    ("temp", "count", "weather", "Temp & Weather vs Bike Rentals")
]

for ax, (x_col, y_col, hue_col, title) in zip(axes, plot_config):
    sns.scatterplot(
        data=df,
        x=x_col,
        y=y_col,
        hue=hue_col,
        ax=ax
    )

    ax.spines[["top", "right"]].set_visible(False)
    ax.set_xlabel("Temperature")
    ax.set_ylabel("Bike Rentals")
    ax.set_title(title, pad=15, weight="bold")
    ax.legend(frameon=False)
```



Temp vs Count vs Season

- Spring temperatures are mostly distributed between approximately 0–30°C, with rental counts generally increasing as temperature increases.
- Winter temperatures are mostly distributed between approximately 5–30°C, with rental counts generally increasing with temperature. However, its average rental demand is lower than fall and summer.
- Summer temperatures are distributed approximately between 10–40°C. Rental counts generally increase with temperature, with summer having the second-highest average rental demand after fall.
- Fall shows the highest average rental demand among all seasons, with many high-count observations concentrated around approximately 25–35°C. Its temperature range appears narrower than summer's.
- The relationship between temperature and rental demand varies across seasons. Fall contains a greater concentration of high-demand observations, while spring generally shows lower rental demand at comparable temperatures. This suggests that season may provide additional context to the relationship between temperature and rental demand.

Temp vs Count vs Weather

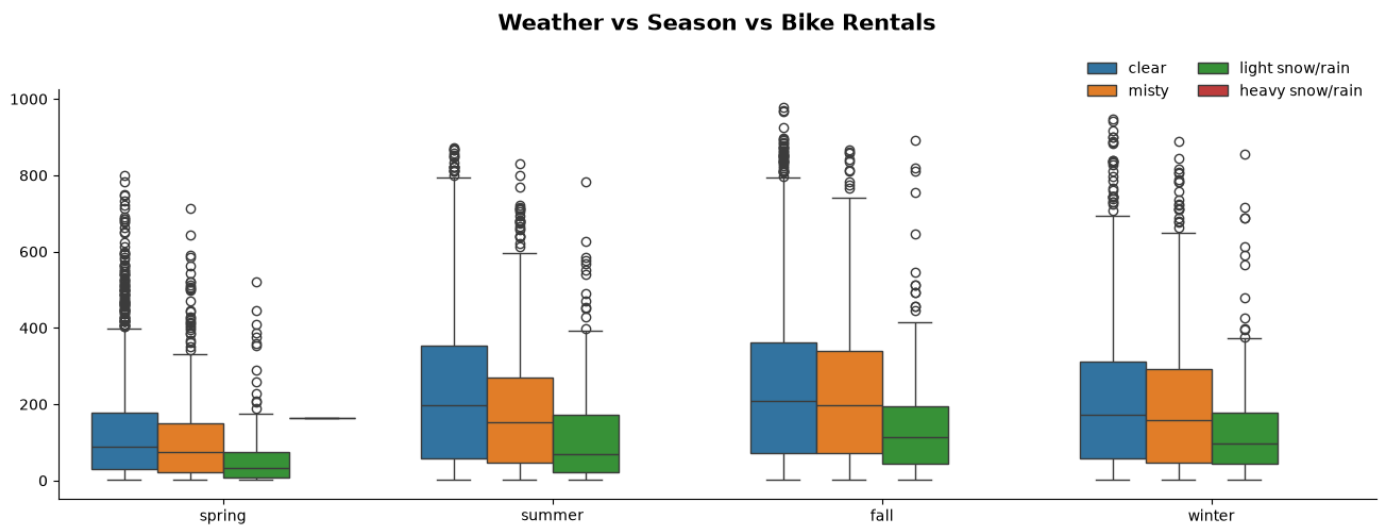
- Weather category 1 (Clear/Partly Cloudy) contains the majority of observations and covers the widest temperature range, approximately 0–40°C. It also contains many of the higher rental-count observations.
- Weather category 2 (Mist/Cloudy) has fewer observations than category 1 but still contains a substantial number of higher rental-count observations across a relatively wide temperature range.
- Weather category 3 (Light Rain/Light Snow) has considerably fewer observations and generally lower rental counts compared with weather categories 1 and 2.
- For weather categories 1 and 2, higher rental counts are concentrated around moderate-to-high temperatures, whereas category 3 generally shows lower rental counts even across overlapping temperature ranges.

Categorical vs Categorical vs Numerical

```
In [55]: fig, ax = plt.subplots(figsize=(16,5))

sns.boxplot(data=df, x="season", y="count", hue="weather")
ax.set_title("Weather vs Season vs Bike Rentals", pad=40, weight="bold", fontsize=15)
ax.spines[["top", "right"]].set_visible(False)
ax.legend(ncol=2, frameon=False, bbox_to_anchor=(1.0,1.1))
ax.set(xlabel=None, ylabel=None)
```

```
plt.show()
```



Key findings

- Rental demand differs significantly across weather conditions.
- Rental demand differs significantly across seasons, with Fall showing the highest average demand and Spring the lowest.
- No statistically significant difference was found in average rental demand between working and non-working days.
- Season and weather are statistically associated.
- Temperature shows a moderate positive association with rentals, while humidity shows a moderate negative association.
- Hourly and user-category analysis reveals additional demand patterns useful for operational planning.

Business Recommendations

- Demand is highest in fall and also strong in summer yulu should plan to improve to position more bikes and operational capacity during these high demand periods.
- Weather forecast can be used as Clear and Misty weather shows higher demand than light snow/rain and heavy snow/rain.
- Lower demand weathers can be used for maintenance and discounts.
- Bikes can be moved to high demand location in early morning hours and in afternoon since rental demand peaked during those periods.
- Since moderate temperatures are associated with stronger demand while very high humidity is associated with lower rental counts. Weather based notifications could timed according to favourable riding conditions.
- Subscription oriented strategies can be used since registered riders dominate more than casual riders accross months.